Rebuilding a Small Transformer from Scratch: Measured Foundations and a Neuro-Symbolic Design for the Reasoning Gap

A Progress Report

Ned Pham
School of Computer Science, University of Nottingham, UK

***Abstract*—We rebuild, without machine-learning libraries, the chain of ideas that leads to a GPT-style language model: a rule-based chatbot, a subword tokenizer, a network trained by back-propagation, word embeddings, and a decoder-only transformer. This paper reports what is finished and measured, and what is not. Stages 0–3 run and are tested; we report XOR learning and Skip-gram embeddings over four random seeds. The seeds reach the same function with very different weights: Skip-gram vectors for the same word differ across seeds (cosine between -0.28 and 0.09) while the pairwise geometry agrees (Pearson $r = 0.977$–0.984). A cross-entropy objective rewards such fits but contains no term for why an output is right. We use the taxonomy of Sheth, Roy and Gaur to propose a feasible neuro-symbolic extension: a logit-level knowledge constraint and a decoupled verifier with a reasoning trace. The transformer and the symbolic layer are *not yet implemented*, so we report no results for them; their experiments are specified as falsifiable predictions. Every number in this paper is regenerated by one command.**

***Index Terms*—transformer, self-attention, neuro-symbolic AI, word embeddings, reproducibility, reasoning**

I. Introduction

Large language models predict the next token. A study note that motivates this work puts the consequence bluntly: the model "guess[es] the best weight to achieve target error" but lacks critical reasoning about its own output, its behaviour is "pure emergent" rather than "designed", and neuro-symbolic AI is named as the route out of the black box [1]. We take that statement as a hypothesis and ask two questions. (i) What does fitting random weights to a loss actually produce in a model small enough to inspect? (ii) What is the smallest neuro-symbolic extension that could be built and tested on such a model?

Contributions. (1) A working, seeded, tested from-scratch pipeline for stages 0–3, including the diagnosis and repair of a collapsed Skip-gram implementation (Sec. IV). (2) A measurement of seed sensitivity: same function, different weights (Sec. IV). (3) A transformer specification that follows the note's order and adds the equations the note omits (Sec. V). (4) A scoped neuro-symbolic design mapped onto the taxonomy of [2], with falsifiable predictions and stated limits (Sec. VII).

What this paper does not contain. It contains no trained-transformer metric, no attention visualisation, and no symbolic-layer result, because those components do not exist yet. Figures 1–5 are schematics and are labelled as such.

II. Background and Related Work

Transformer. Vaswani et al. [3] replace recurrence with scaled dot-product attention. We follow the pre-layer-norm decoder-only variant [4], trained with Adam [5]. *Earlier stages.* ELIZA [6] is the rule-based baseline; byte-pair encoding [7] gives the subword vocabulary; back-propagation [8] trains the XOR network; Skip-gram with negative sampling [9], [10] gives static embeddings.

Neuro-symbolic AI. Sheth, Roy and Gaur [2] divide methods into *lowering* (Category 1: compress symbolic knowledge for use inside a neural network) and *lifting* (Category 2: lift neural patterns into symbolic form). Each splits in two, giving four subcategories (Fig. 6). The paper does not use Kautz's six patterns and does not use the terms "learning for reasoning" or "reasoning for learning"; we do not either.

Reference design. Our architecture cross-checks against a public tutorial repository [12]. It has no licence file, so we copied no code, and it contains no symbolic component.

III. System and Reproducibility Protocol

Fig. 1 shows the pipeline and the status of each stage. The stack is TypeScript on Bun with no ML libraries. All randomness goes through one seeded generator (Mulberry32), so a run is a pure function of its seed.

Reproduction. `bun tests/ieee-data.ts` regenerates every CSV, JSON and LaTeX fragment in `docs/progress/ieee/data/`; `main.tex` reads only those files. `bun test tests/train-embed.test.ts` runs the Skip-gram sanity tests. Seeds are $\{42, 1, 2, 3\}$. A second run of the same seed returned an identical final loss for the XOR network.

IV. Measured Results, Stages 2–3

A. XOR: same function, different weights

A single-layer perceptron cannot represent XOR; a $2 \to 4 \to 1$ sigmoid network trained by back-propagation can [8]. Loss is mean-squared error over the four inputs, learning rate 1.0, full-batch.

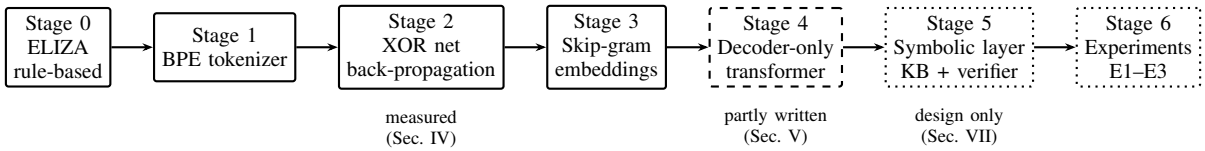


Fig. 1. Pipeline and status. Solid border: implemented and measured. Dashed: in progress. Dotted: specified, not built. Stage 0–1 are exercised by the application but contribute no metric to this paper.

TABLE I
XOR, $2 \to 4 \to 1$ NETWORK, FOUR SEEDS. OUTPUTS ARE FOR INPUTS $(0,0), (0,1), (1,0), (1,1)$ AFTER 5000 EPOCHS; THE LAST COLUMN IS THE FIRST ROW OF THE INPUT-TO-HIDDEN WEIGHTS AFTER 5000 EPOCHS.

Seed	MSE@500	MSE@5000	Outputs	$w_1[0,:]$
42	0.0144	2.8e−4	0.01 / 0.98 / 0.98 / 0.02	$[5.8, -1.5, 6.2, -0.3]$
1	0.0227	2.7e−4	0.01 / 0.98 / 0.98 / 0.02	$[0.0, -6.5, -1.9, 5.3]$
2	0.1506	3.1e−4	0.01 / 0.98 / 0.98 / 0.02	$[3.5, -2.6, -6.7, -5.2]$
3	0.0110	2.2e−4	0.01 / 0.98 / 0.99 / 0.01	$[4.0, -2.9, 1.2, -6.6]$
single-layer (42)	0.2842	–	0.53 / 0.50 / 0.47 / 0.43	–

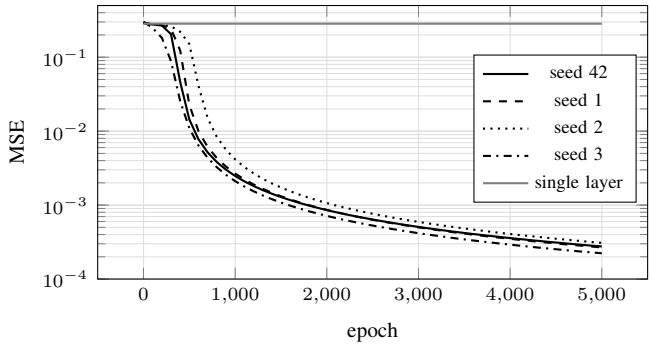


Fig. 2. XOR training loss. The $2 \to 4 \to 1$ network (four seeds) reaches $\approx 10^{-4}$; the single-layer perceptron stays at 0.2842 (from 0.2845) and outputs near 0.5.

TABLE II
SKIP-GRAM AFTER 300 EPOCHS, FOUR SEEDS. COSINES ARE BETWEEN INPUT VECTORS. ANALOGY ACCURACY USES OUR OWN ONE-ANSWER-PER-QUERY KEY (SEC. VIII).

Seed	Loss	k/q	m/w	c/d	c/k	Analogy
42	4.074 → 1.271	0.90	0.92	0.72	0.47	6/7
1	4.080 → 1.261	0.88	0.92	0.73	0.47	5/7
2	4.068 → 1.266	0.89	0.91	0.70	0.56	5/7
3	4.078 → 1.244	0.88	0.91	0.70	0.47	6/7

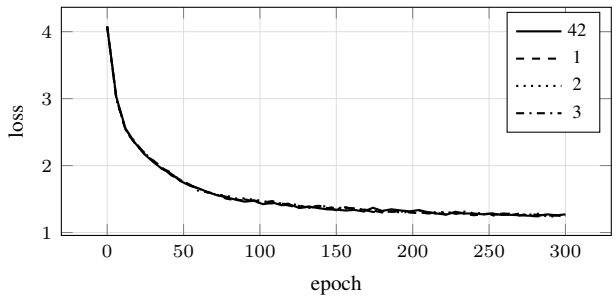


Fig. 3. Skip-gram negative-sampling loss, four seeds.

All four seeds solve XOR at 5000 epochs, yet the first-layer weights differ in sign and magnitude (Table I). At 500 epochs the seeds are not equivalent: seed 2 has not finished (MSE 0.1506 versus 0.0110 for seed 3). This is the smallest instance of the note's point: the fitted weights are an accident of initialisation, and only the input–output function is stable.

B. Skip-gram: geometry agrees, coordinates do not

The corpus has 107 sentences, a vocabulary of 191 words and 1878 (target, context) pairs (window 2, 32 dimensions, 5 negatives, 300 epochs). An earlier version of the trainer collapsed: every cosine similarity was ≈ 1.0. We found five defects (an off-by-one in the cumulative sampling table, a missing positive-pair update, the wrong row updated for negatives, an inverted analogy sign, and a malformed corpus join) and fixed them; the sanity tests in `tests/train-embed.test.ts` guard against recurrence.

Across seeds the *same word* has unrelated vectors: the mean cosine between a word's vectors from two seeds ranges from -0.28 to 0.09 over the six seed pairs. Yet the pairwise-cosine matrices agree: Pearson r between two seeds' matrices lies in $[0.977, 0.984]$ (Fig. 4). Coordinates are arbitrary; relations are not. This refines the note's claim: weights are not reproducible, but the learned function is, at this scale. It does not show that the function is *correct*.

A fitted answer that is confidently wrong. For seed 42, the query "kitten − cat + dog" returns *lion* with cosine 0.85; the key expects *puppy*. Seeds 1–3 return *lion* as well. The score measures similarity to a fitted vector, not support for the inference; the objective has no term that would distinguish the two. This is one measured instance of the reasoning gap discussed in Sec. VI.

V. THE TRANSFORMER (SPECIFIED, NOT YET TRAINED)

The note walks from token-parallel attention heads, through output projection and a feed-forward step, to the transformer

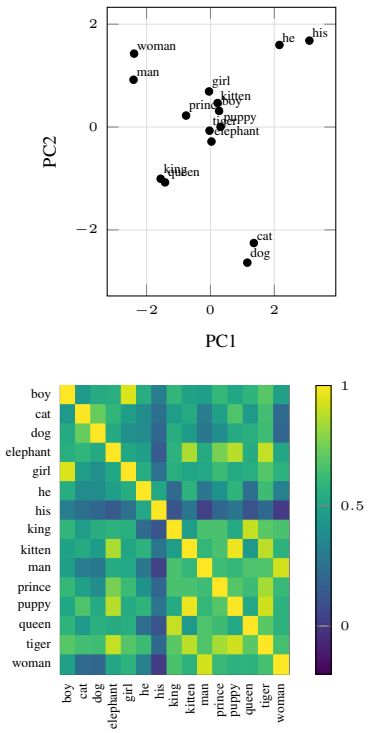


Fig. 4. Skip-gram, seed 42. Top: 2-D PCA of the 15 queried word vectors (from 32 dimensions; axes carry no meaning). Bottom: pairwise cosine matrix.

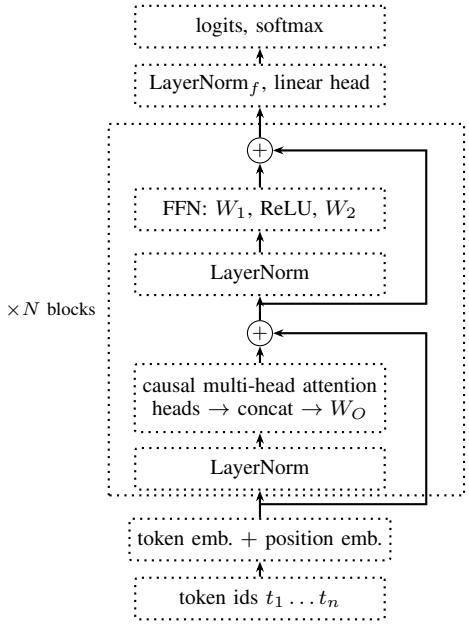


Fig. 5. Designed decoder-only block (pre-LN), after the study note's steps 3–5 [1] and Vaswani et al. [3]. Attention is $\mathrm{softmax}(QK^\top/\sqrt{d_k}+M)V$ per head, Eq. (2). Dotted: specified, not trained.

block [1]. Fig. 5 draws that path as our design. The code (`train-transformer/transformer.ts`) defines the types, configuration and initialisation but lacks forward, backward and Adam updates and does not compile; a list of known defects is in the repository report.

Equations (added; the note has none). With X the sequence of embeddings,

$$Q = XW_Q + b_Q,\; K = XW_K + b_K,\; V = XW_V + b_V \tag{1}$$

$$\mathrm{Attn}(X) = \mathrm{softmax}\Big(\frac{QK^\top}{\sqrt{d_k}} + M\Big)V,\; M_{ij} = -\infty\;(j > i) \tag{2}$$

$$\mathcal{L}(\theta) = -\tfrac{1}{n}\textstyle\sum_i \log p_\theta(t_{i+1} \mid t_{\le i}) \tag{3}$$

For $D = 32$ and $F = 128$, a pre-LN block has $4(D^2{+}D) + 2(2D) + (DF{+}F) + (FD{+}D) = 12{,}704$ parameters (analytic count from the block definition; not read from a trained model).

VI. The Reasoning Gap

We separate what is measured from what is argued.

Measured. (a) Different seeds give different weights for the same function (Table I, Fig. 4). (b) A fitted model can return a wrong answer with a high score (Sec. IV).

By construction. Eq. (3) depends on the predicted distribution only; it contains no term for evidence, rule or justification. A model that minimises it can be right for the wrong reason, and nothing in training detects this. This is a property of the objective, not an experimental finding.

Not shown. We have no reasoning benchmark and no trained transformer, so we do not claim that attention models *fail* at reasoning, nor how a small model compares with a large one. Our corpus is tiny; it demonstrates the mechanism, not scale behaviour. Stability of the function across seeds (Sec. IV) is evidence *against* a strong reading of "random" and is reported as such.

VII. Neuro-Symbolic Design (Not Implemented)

A. Taxonomy mapping

Fig. 6 redraws Fig. 1 of [2], including its low/medium/high ratings, and marks where each proposed component sits. A logit mask or bias is a *lowering*-style intervention only by analogy: [2] compresses knowledge into embeddings or logic representations, while we apply it to the output scores. A verifier with a trace is a decoupled *lifting* method (2a). A training-time semantic loss [11] would move toward 2b and is future work.

B. Mechanisms and the code seam

Fig. 7 shows the intended data flow. A knowledge base of triples and Horn-style rules is expanded by forward chaining. A *logits processor* receives a fresh copy of the length-V logit vector before temperature and softmax: banned tokens are set to $-\infty$ (which stays $-\infty$ after temperature and top-p), supported tokens receive $+\lambda$, and if every token would be banned the processor falls back to the unmasked logits and records that in the trace. Each step logs {rules fired, masked tokens, probability mass before and after, chosen token}. A decoupled verifier checks the final text against the same knowledge base.

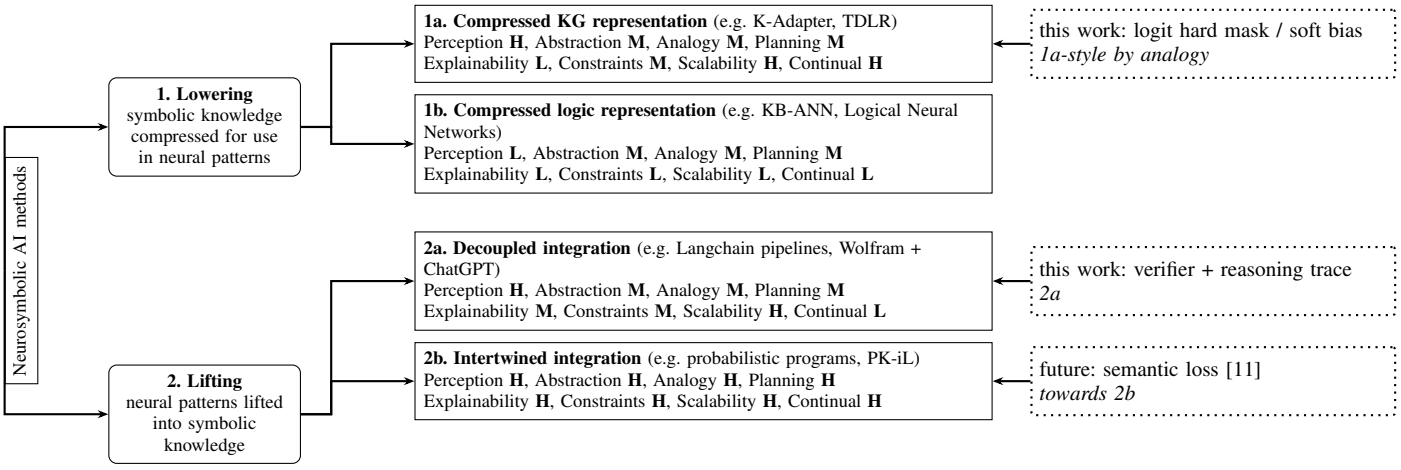


Fig. 6. Taxonomy of [2] (redrawn from its Fig. 1; L/M/H = low/medium/high as rated by those authors) with the position of each component in this work. Dotted boxes are our mapping, not the original authors'.

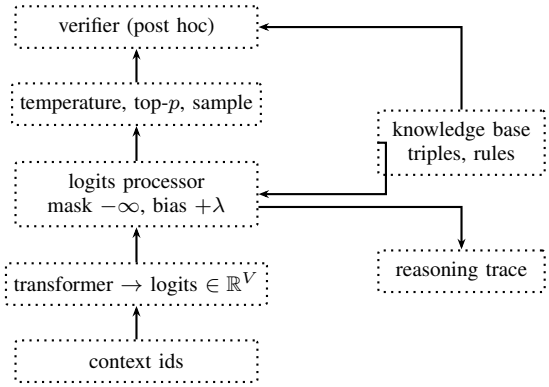


Fig. 7. Proposed neuro-symbolic data flow. Nothing in this figure is implemented.

C. Falsifiable predictions and protocol

Experiments are *specified*, not run. **E1** (seed sensitivity): train the transformer under four seeds; report final loss spread and weight-space distance. **E2** (constraint violations): generate N samples per seed with the processor off and on; report KB-violation rate, perplexity and distinct-token ratio. **E3**: one generation with its full trace. Predictions that would be *refuted* by a bad result: P1, with the hard mask on the violation rate for token-expressible constraints is 0; P2, with the processor off, generation is byte-identical to the unconstrained baseline for the same seed; P3, loss spread across seeds is small while weight distance is large; P4, a soft bias lowers violations without eliminating them. P1–P2 are properties of the design and can fail only through bugs; P3–P4 are empirical.

VIII. Threats to Validity and Limitations

- *Scale.* 107 sentences and 191 words. Results demonstrate mechanisms, not scale behaviour.
- *Few seeds, no tests.* Four seeds per configuration, no confidence intervals or significance tests; ranges are reported as observed.
- *Answer key.* The analogy key is one answer per query chosen by us; alternative acceptable answers would change the accuracy column.
- *No held-out evaluation.* Skip-gram is judged on the training corpus; XOR has four points by definition.
- *Single configuration.* One dimensionality, window and learning rate. No ablation.
- *Symbolic constraints act on tokens.* Words split into several subword tokens can be constrained only on their first token.
- *Saved artefact.* The tracked `.data/embedding-weights.json` still holds the earlier, collapsed model; the numbers here come from fresh runs with saving off.

IX. Conclusion and Future Work

Stages 0–3 are repaired and measured. Their seeds agree on function and disagree on weights, which supports a refined form of the note's observation. The hypothesis that fitted weights lack justification is, in Eq. (3), true by construction, and the wrong-but-confident analogy illustrates it; whether a symbolic layer closes the gap is untested. The next steps, in order, are to finish the transformer (forward, backward, Adam) with finite-difference gradient checks; implement the logits processor and verifier; run E1–E3; and replace this paper's "not implemented" statements with measured results.

Provenance

Fig. 6 is redrawn from [2]; Fig. 5 follows the study note [1]. No figure is a copy of a published figure, and no code from the reference repository [12] was copied.

References

[1] N. Pham, "NLP Transformer Note," unpublished study note, 5 pp., 2026 (`docs/paper/`).
[2] A. Sheth, K. Roy, and M. Gaur, "Neurosymbolic artificial intelligence (why, what, and how)," *IEEE Intell. Syst.*, vol. 38, no. 3, pp. 56–62, May/Jun. 2023.
[3] A. Vaswani *et al.*, "Attention is all you need," in *Proc. NeurIPS*, 2017.

[4] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer normalization,” arXiv:1607.06450, 2016.
[5] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. ICLR*, 2015.
[6] J. Weizenbaum, “ELIZA—a computer program for the study of natural language communication between man and machine,” *Commun. ACM*, vol. 9, no. 1, pp. 36–45, 1966.
[7] R. Sennrich, B. Haddow, and A. Birch, “Neural machine translation of rare words with subword units,” in *Proc. ACL*, 2016, pp. 1715–1725.
[8] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
[9] T. Mikolov, K. Chen, G. Corrado, and J. Dean, “Efficient estimation of word representations in vector space,” arXiv:1301.3781, 2013.
[10] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean, “Distributed representations of words and phrases and their compositionality,” in *Proc. NeurIPS*, 2013.
[11] J. Xu, Z. Zhang, T. Friedman, Y. Liang, and G. Van den Broeck, “A semantic loss function for deep learning with symbolic knowledge,” in *Proc. ICML*, 2018.
[12] w3cj, “how-llms-work,” GitHub repository, https://github.com/w3cj/how-llms-work (no licence file; architecture cross-check only).